Requirement Analysis

RUP and **UML**

Requirement Analysis

A **requirement** is defined as a condition or capability to which a system must conform. Requirements management is a systematic approach to finding, documenting, organizing and tracking the changing requirements of a system.

There are many different kinds of requirements. Requirements are usually looked upon as statements of text fitting into one of the following categories:

1. **User requirements:** User requirements focus on the needs and goals of the end users. They are statements of what services the system is expected to provide and the constraints under which it must operate.
2. **System requirements:** They set out the system's functions, services and operational constraints in detail.

Requirement Analysis

Software system requirements are often classified as functional and non-functional requirements:

1. **Functional requirements:** They specify actions that a system must be able to perform, without taking physical constraints into consideration. Functional requirements thus specify the input and output behavior of a system.
2. **Non-functional requirements**: Many requirements are non-functional, and describe only attributes of the system or attributes of the system environment. They are often categorized as usability, reliability, performance, and substitutability requirements. They are often requirements that specify need of compliance with any legal and regulatory requirements. They can also be design constraints due to the operating system used, the platform environment, compatibility issues, or any application standards that apply.

Domain requirements are a subcategory of requirements.

These are requirements that come from the application domain of the system and that reflect characteristics and constraints of that domain.

They are formulated by domain-specific terminology. They may be functional or non-functional requirements.

The user requirements are provided by functional requirements. The functional requirements are often best described in a UML use-case model and in use cases. Every functionality of system is modelled by one or more use cases.

The general objectives of the requirements discipline is:

1. To establish and maintain agreement with the customers and other stakeholders on what the system should do.
2. To provide system developers with a better understanding of the system requirements.
3. To define the boundaries of the system.
4. To provide a basis for planning the technical contents of iterations.
5. To provide a basis for estimating cost and time to develop the system.
6. To define a user-interface for the system, focusing on the needs and goals of the users.

Requirement Analysis

In the frame of RUP methodology the activities and artifacts are organized into workflow details as follows:

1. Analyze the Problem
2. Understand Stakeholder Needs
3. Define the System
4. Manage the Scope of the System
5. Refine the System Definition
6. Manage Changing Requirements

Each workflow detail represents a key skill that need to be applied to perform effective requirements management. Analyze the Problem and Understand Needs are focused on during the Inception phase of a project, whereas the emphasis is on Define the System and Refine the System Definition during the Elaboration phase. Manage the Scope of the System and Manage Changing Requirements are done continuously throughout the development project.

The workflow details are listed above in the sequence that is most likely applied to the first iteration of a new project. They are applied continuously in varied order as needed throughout the project.

Requirement Analysis

1. Analyse The Problem

Problem analysis is done to understand problems, initial stakeholder needs, and propose high-level solutions. It is an act of reasoning and analysis to find "the problem behind the problem".

During problem analysis, agreement is gained on the real problems and stakeholders are identified. Also, initial boundaries of the solution and constraints are defined from both technical and business perspectives.

The purpose of this workflow detail is to:

1. Gain agreement on the problem being solved,
2. Identify stakeholders,
3. Define the system boundaries, and
4. Identify constraints imposed on the system.

Requirement Analysis

2. Understand Stakeholder Needs

The purpose of this workflow detail is to elicit and collect information from the stakeholders of the project in order to understand what their needs are. The collected stakeholder requests can be regarded as a "wish list" that will be used as primary input to defining the high-level features of our system, as described in the Vision. This activity is mainly performed during iterations in the inception and elaboration phases.

The key activity of this workflow is to elicit stakeholder requests using such input as business rules, enhancement requests, interviews and requirements workshops. The primary outputs are collections of prioritized features and their critical attributes.

This information results in a refinement of the Vision document, as well as a better understanding of the requirements attributes. Another important output is an updated Glossary of terms to facilitate common vocabulary among team members.

Requirement Analysis
3. Define the system

The purpose of this workflow detail is to:

1. Align the project team in their understanding of the system.
2. Perform a high-level analysis on the results of collecting stakeholder requests.
3. Refine the Vision to include the features to include in the system, along with appropriate attributes.
4. Refine the use-case model, to include outlined use cases.
5. More formally document the results in models and documents.

Requirement Analysis
3. Define the system

Problem Analysis and activities for Understanding Stakeholder Needs create early iterations of key system definitions, including the features defined in the Vision document, a first outline to the use-case model and the Requirements Attributes. In Defining the System you will focus on identifying actorsand use cases more completely, and expand the global non-functional requirements as defined in the Supplementary Specifications. Refining and structuring the use-case model has three main reasons for:

1. To make the use cases easier to understand.
2. To partition out common behaviour described within many use cases
3. To make the use-case model easier to maintain.

Requirement Analysis

3. Define the system

An aspect of organizing the use-case model for easier understanding is to group the use cases into packages. A model structured into smaller units is easier to understand. It is easier to show relationships among the model's main parts if they are expressed in terms of packages. The use-case model can be organized as a hierarchy of use-case packages, with "leaves" that are actors or use cases.

A use-case package contains a number of actors, use cases, their relationships, and other packages; thus, there may be a multiple levels of use-case packages. Use-case packages can reflect order, configuration, or delivery units in the finished system. The packages can be organised according the principles:

1. A package of use cases can be related to a subsystem.
2. A package of use cases can be related to an actor.
3. Packages can be formed by any logical grouping.

Requirement Analysis

4. Manage the scope of the system

The objectives of this workflow detail are to:

1. Prioritize and refine input to the selection of features and requirements that are to be included in the current iteration.
2. Define the set of use cases (or scenarios) that represent some significant, central functionality.
3. Define which requirement attributes and traceabilities to maintain.

The scope of a project is defined by the set of requirements allocated to it. Managing project scope, to fit the available resources such as time, people, and money is key to managing successful projects. Managing scope is a continuous activity that requires iterative or incremental development, which breaks project scope into smaller more manageable pieces.

Using requirement attributes, such as priority, effort, and risk, as the basis for negotiating the inclusion of a requirement is a particularly useful technique for managing scope.

Requirement Analysis

5. Refine the system definition

The purpose of this workflow detail is to further refine the requirements in order to:

1. Describe the use case's flow of events in detail.
2. Detail Supplementary Specifications.
3. Develop a Software Requirements Specification, if more detail is needed, and
4. Model and prototype the user interface.

The output of this workflow detail is a more in-depth understanding of system functionality expressed in detailed use cases, revised and detailed Supplementary Specifications, as well as user-interface elements. A formal Software Requirements Specification may be developed, if needed, in addition to the detailed use cases and Supplementary Specifications.

It is important to work closely with users and potential users of the system when modelling and prototyping the user-interface. This may be used to address usability of the system, to help uncover any previously undiscovered requirements and to further refine the requirements definition.

Quiz

Q1. Which of these are functional requirements?

1. Users of the library are either normal or staff
2. A user can borrow a book
3. A staff person can borrow a book
4. The library contains one million books
5. If a user asks a book that has been borrowed, her request is inserted in a waiting list
6. Staff has no priority in borrowing books

Which of these are non-functional requirements?

1. Pressing the switch the room gets lightened
2. Pressing the switch the room gets lightened in less than one second
3. If the room is dark, pressing the switch it gets lightened
4. The light in the room must be sufficient to read
5. If someone is reading then the light must stay on
6. After two minutes that the room is empty the light must switch off

Rational Unified Process

RUP

Rational Unified Process - **RUP**

- The Rational Unified Process is a Software Engineering Process.
- It provides a disciplined approach to assigning tasks and responsibilities within a development organization.
- Its goal is to ensure the production of high-quality software that meets the needs of its end-users, within a predictable schedule and budget.
- The fundamental purpose of the Rational Unified Process is to provide a model for effectively implementing commercially proven approaches to development, for use throughout the entire software development life cycle.
- Taking elements from other iterative software development models, the Rational Unified Process framework was initially created by the Rational Software Corporation, which was bought out by IBM in 2003.

Rational Unified Process - **RUP**

- The Rational Unified Process is not a concrete development model, but rather is intended to be adaptive and tailored to the specific needs of your project, team, or organization.
- The Rational Unified Process is based on a few fundamental ideas, such as the phases of development and the building blocks, which define who, what, when, and how development will take place.

RUP - Best Practices

The Rational Unified Process is structured around six fundamental best practices, which are so-named due to their common use throughout the industry

RUP - Best Practices

1. **Develop Software Iteratively**: Encourages iterative development by locating and working on the high-risk elements within every phase of the software development life cycle.
2. **Manage Requirements**: Describes how to organize and keep track of functionality requirements, documentation, tradeoffs and decisions, and business requirements.
3. **Use Component-Based Architectures**: Emphasizes development that focuses on software *components* which are reusable through this project and, most importantly, within future projects.
4. **Visually Model Software**: Based on the Unified Modeling Language (UML), the Rational Unified Processprovides the means to visually model software, including the components and their relationships with one another.
5. **Verify Software Quality**: Assists with design, implementation, and evaluation of all manner of tests throughout the software development life cycle.
6. **Control Changes to Software**: Describes how to track and manage all forms of change that will inevitably occur throughout development, in order to produce successful iterations from one build to the next.

RUP - The Building Blocks

All aspects of the Rational Unified Process are based on a set of building blocks, which are used to describe **what** should be produced, **who** is in charge of producing it, **how** production will take place, and **when** production is complete. These four building blocks are:

- **Workers, the 'Who'**: The behavior and responsibilities of an individual, or a group of individuals together as a team, working on any activity in order to produce artifacts.
- **Activities, the 'How'**: A unit of work that a worker is to perform. Activities should have a clear purpose, typically by creating or updating artifacts.
- **Artifacts, the 'What'**: An artifact represents any tangible output that emerges from the process; anything from new code functions to documents to additional life cycle models.
- **Workflows, the 'When'**: Represents a diagrammed sequence of activities, in order to produce observable value and artifacts.

RUP - The Building Blocks

Workflows are further divided up in the Rational Unified Process into six core engineering workflows:

- **Business Modeling Workflow**: During this workflow, the business context (scope) of the project should be outlined.
- **Requirements Workflow**: Used to define all potential requirements of the project, throughout the software development life cycle.
- **Analysis & Design Workflow**: Once the requirements workflow is complete, the analysis and design phase takes those requirements and transforms them into a design that can be properly implemented.
- **Implementation Workflow**: This is where the majority of actual coding takes place, implementing and organizing all the code into layers that make up the whole of the system.
- **Test Workflow**: Testing of all kinds takes place within this workflow.
- **Deployment Workflow**: Finally, the deployment workflow constitutes the entire delivery and release process, ensuring that the software reaches the customer as expected.

RUP - The Building Blocks

There are also three core supporting workflows defined in the Rational Unified Process:

- **Project Management Workflow**: Where all activities dealing with project management take place, from pushing design objectives to managing risk to overcoming delivery constraints.
- **Configuration & Change Management Workflow**: Used to describe the various artifacts produced by the development team, ideally ensuring that there is minimal overlap or wasted efforts performing similar activities that result in identical or conflicting artifacts.
- **Environment Workflow**: Finally, this workflow handles the setup and management of all software development environments throughout the team, including the processes, as well as the tools, that are to be used throughout the software development life cycle.

RUP - The Four Life Cycle Phases

The four life cycle phases in RUP are:

1. Inception
2. Elaboration
3. Construction
4. Transition

RUP - Inception Phase

The main goal of the Inception phase is to achieve concurrence among all stakeholders on the lifecycle objectives of the project. The following are the primary Inception phase objectives:

- To establish the project's scope and boundary conditions
- To identify the critical use cases of the system
- To exhibit and demonstrate one candidate architecture
- To estimate the overall cost and schedule for the project
- To produce detailed estimates for the Elaboration phase
- To estimate the potential risks
- To prepare the support environment for the project

The RUP is risk driven; the highest risks are identified earliest, and efforts are made to mitigate or address those risks as early in the project lifecycle as possible instead of pushing them forward. The Inception phase plays the most critical role in the project and will result in the first release of the product. In such cases, significant business and requirements risks need to be carefully managed. Accordingly, for new releases or enhancements of existing products, the Inception phase becomes much shorter. The Lifecycle Objectives milestone concludes the Inception phase. At that point, a major decision is made on whether to proceed with the project or cancel it.

RUP - Elaboration Phase

The main goal of the Elaboration phase is to baseline the architecture of the system to provide a stable basis for the bulk of the design and implementation effort in the Construction phase. The architecture evolves based on the most significant requirements and assessment of risks. To evaluate the stability of the architecture, one or more architectural prototypes may be developed. This architectural prototype is the executable architecture. The Elaboration phase objectives are as follows:

- To stabilize the architecture, requirements, and respective plans
- To sufficiently mitigate risks to predictably determine project cost and schedule
- To address all architecturally significant risks
- To establish a baselined architecture
- To produce an evolutionary prototype of production-quality components
- Optionally, to produce throw-away prototypes to mitigate specific risks such as design trade-offs, component reuse, and product feasibility
- To demonstrate that the baselined architecture will support the requirements of the system at a reasonable cost and in a reasonable time
- To establish a supportive environment

The Lifecycle Architecture milestone concludes the Elaboration phase, establishing a managed baseline for the architecture of the system and enabling the project team to scale during the Construction phase.

RUP - Construction Phase

The main goals of the Construction phase are to clarify the remaining requirements and complete the development of the system based on the baselined architecture. Construction phase objectives can be briefly summarized as follows:

- To minimize development costs through optimization of resource utilization by avoiding unnecessary scrap and rework and by achieving a degree of parallelism in the work of development teams
- To achieve adequate quality as rapidly as is practical
- To achieve useful executable versions (alpha, beta, and so on) as rapidly as practical
- To complete the analysis, design, development, and testing of all required functionality
- To iteratively and incrementally develop a complete product that is ready to transition to its user community
- To decide if the software, the sites, and the users are ready for the deployment of the solution

The Construction phase concludes with the Initial Operational Capability milestone, which determines whether the product is ready to be deployed into a beta-test environment.

RUP - Transition Phase

The overall goal of the Transition phase is to ensure that software is available for its users. It can span several iterations and includes testing the product in preparation for release and making minor adjustments based on user feedback. This feedback focuses primarily on fine-tuning the product, configuration, installation, and usability issues. All the major structural issues should have been worked out much earlier in the project lifecycle. Following are the primary objectives of the Transition phase:

- To validate the new system against user expectations (by beta testing)
- To train the end users and maintainers
- If applicable, to roll out the product to marketing, distribution, and sales teams
- To fine-tune the product by engaging in bug-fixing and creating performance and usability enhancements
- To conclude the assessment of the deployment baseline against the complete vision and the acceptance criteria for the product
- To achieve user self-supportability
- To achieve stakeholder concurrence that deployment baselines are complete and are consistent with the evaluation criteria of the vision

The Product Release milestone concludes this phase. A decision is made whether the objectives of the project were met.

RUP - Iterations

- The Rational Unified Process also recommends that each of the four above phases be further broken down into iterations, a concept taken from agile and other common iterative development models.
- Just as with those other models, in the context of the Rational Unified Process, an iteration simply represents a full cycle of the aforementioned core phases, until a product is released in some form (internally or externally).
- From this baseline, the next iteration can be modified as necessary until, finally, a full and complete product is released to customers.

Advantages of **RUP**

1. Allows for the adaptive capability to deal with changing requirements throughout the development life cycle, whether they be from customers or from within the project itself.
2. Emphasizes the need (and proper implementation of) accurate documentation.
3. Diffuses potential integration headaches by forcing integration to occur throughout development, specifically within the construction phase where all other coding and development is taking place.

Disadvantages of **RUP**

1. Heavily relies on proficient and expert team members, since assignment of activities to individual workersshould produce tangible, pre-planned results in the form of artifacts.
2. Given the emphasis on integration throughout the development process, this can also be detrimental during testing or other phases, where integrations are conflicting and getting in the way of other, more fundamental activities.
3. Arguably, rup is a fairly complicated model. Given the assortment of the components involved, including best practices, phases, building blocks, milestone criteria, iterations, and workflows, often proper implementation and use of the Rational Unified Process can be challenging for many organizations, particularly for smaller teams or projects.

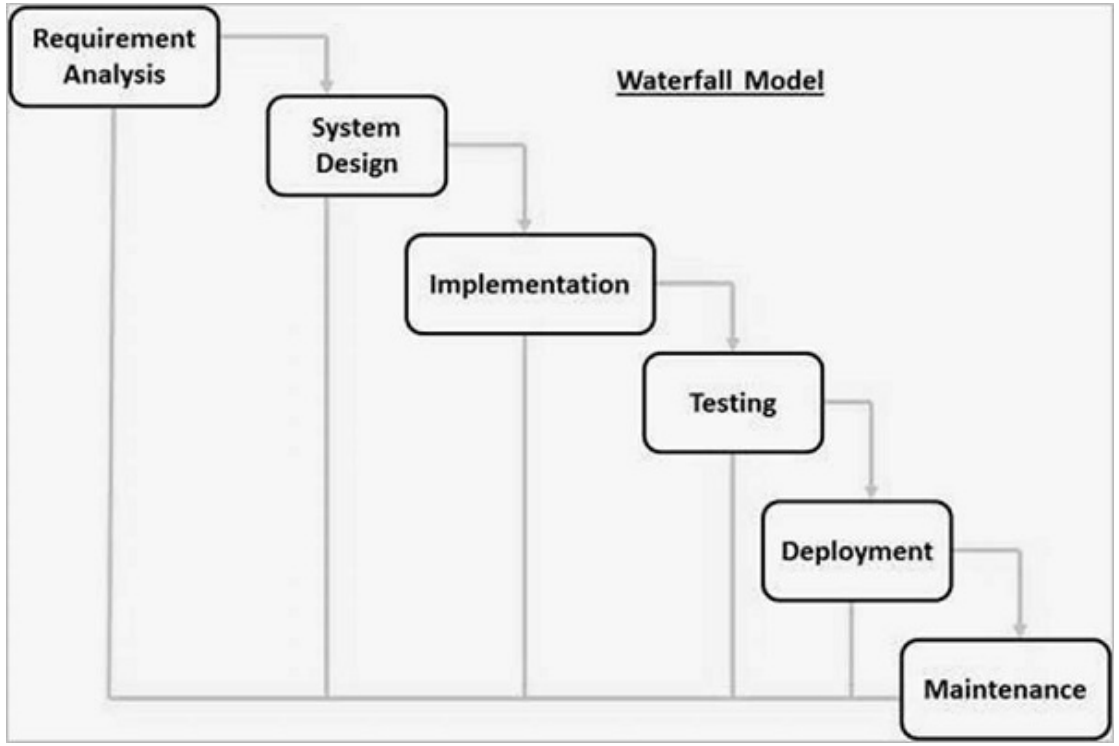
How Does RUP Compare to the Waterfall Model?

What is the Waterfall Model?

- The Waterfall Model was the first Process Model to be introduced. It is also referred to as a **linear-sequential life cycle model**. It is very simple to understand and use. In a waterfall model, each phase must be completed before the next phase can begin and there is no overlapping in the phases.
- The Waterfall model is the earliest SDLC approach that was used for software development.
- The waterfall Model illustrates the software development process in a linear sequential flow. This means that any phase in the development process begins only if the previous phase is complete. In this waterfall model, the phases do not overlap.

Waterfall Model - Design

- Waterfall approach was first SDLC Model to be used widely in Software Engineering to ensure success of the project.
- In "The Waterfall" approach, the whole process of software development is divided into separate phases. In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.



The sequential phases in Waterfall model are –

- **Requirement Gathering and analysis** – All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- **System Design** – The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- **Implementation** – With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.
- **Integration and Testing** – All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
- **Deployment of system** – Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
- **Maintenance** – There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

RUP Phases versus Waterfall Phases

Waterfall Phase Characteristics	RUP Phase Characteristics
In any given phase, the activities are performed from a single area of concern. For example, during the Requirements phase, all activities related to requirements gathering and analysis are performed. No code is produced and no testing is carried out.	In any given RUP phase, depending on which phase it is, there will be activities from across multiple disciplines. For example, during the Elaboration phase, activities performed normally span all the core disciplines, including Requirements, Analysis and Design, Implementation, Test, and others.

RUP Phases versus Waterfall Phases

Waterfall Phase Characteristics	RUP Phase Characteristics
Not all phases result in an executable deliverable. In fact, only Implementation and Test phases may produce executable deliverables.	With the exception of early Inception iterations, each iteration within each phase produces an executable deliverable.
A given waterfall phase employs a subset of team members who are skilled to perform related activities. This might lead to less than optimal resource utilization.	Producing an executable deliverable at the end of most iterations within RUP phases requires activities from across multiple disciplines to be performed and therefore engages the entire team.

RUP Phases versus Waterfall Phases

Waterfall Phase Characteristics	RUP Phase Characteristics
Most waterfall phases result in document-based deliverables.	Most iterations within RUP phases result in an executable deliverable.

RUP - Disciplines

RUP - Disciplines

A discipline is defined as a categorization of activities based on similarity of concerns and cooperation of work effort. A discipline is a collection of activities that are related to a major "area of concern" (or "a field of study," as discussed earlier) within the overall project.

In RUP, an **activity** is a process element that supports the nesting and logical grouping of related process elements, such as a descriptor2 and subactivities, thus forming breakdown structures. The grouping of activities into disciplines is mainly an aid to understanding the project from a traditional waterfall perspective; that is, in a traditional waterfall project, your phases are called Requirements, Analysis, Design, Implementation, Testing, and so on. Therefore, within a waterfall project, you focus on a single discipline and associated artifacts for that discipline. For instance, when you have finished the Requirements phase of a waterfall project, you will gain final approval from the customer and move on to the next phase which, in most cases, is Analysis. In the RUP, although it is more common to perform activities concurrently across several disciplines at any given point during the life of a project (for example, certain Requirements activities are performed in close coordination with Analysis and Design activities), separating these activities into distinct disciplines is simply an effective way to organize content, which makes comprehension and learning easier. In addition, because the skill-sets needed to perform the tasks in one area of concern are probably similar, logical grouping of these activities simplifies the way different roles are organized. This enables us to align a small set of roles along discipline lines.

The Role of Disciplines in the Software Engineering Process and the RUP

In the RUP, however, a discipline refers to a specific area of concern (or a field of study, as mentioned earlier) within software engineering. For instance, Analysis and Design is one of the disciplines in RUP, which is itself a field of study and requires dedicated learning and distinct skill-sets. In addition, disciplines in RUP allow you to govern the activities you perform within that discipline. A discipline in RUP gives you all the guidance you require to learn not only when to perform a given activity but also how to perform it. Therefore, disciplines in RUP allow you to bring closely related activities under control.

The Role of Disciplines in the Software Engineering Process and the RUP

We will see in detail how these related activities are governed and performed in an organized manner, not in isolation and not haphazardly. In fact, a recommended sequence should be followed to achieve optimal performance and maximize productivity and predictability. Note that although disciplines propose a recommended sequence of activities, these are truly performed in parallel with activities from other relevant (based on where you are in the project lifecycle) disciplines. When I was engaged in enabling one of the financial institutions to adopt RUP, I had to have separate sessions and workshops with System Analysts, Business Analysts, Project Managers, and others. During those sessions, the discipline workflows really proved helpful from the perspective of the involvement of a given role and the related activities to be performed across the RUP lifecycle. The workflows helped the people with given roles appreciate the effort that was required within their discipline.

The Role of Disciplines in the Software Engineering Process and the RUP

A clear understanding of these relationships between roles and disciplines and the appreciation of how different roles from across different disciplines collaborate throughout the project lifecycle is important. It is crucial that you establish a clear understanding of this concept right from the beginning, and it will be helpful as you become immersed in the iterative development world.

The benefits provided by separating the RUP activities into various disciplines are summarized as follows.

- Makes the activities easier to comprehend.
- Proves useful when customizing a given discipline to meet the specific needs of the project or when defining a set of organizational standard processes.
- Enables different roles to better and more effectively appreciate their responsibilities (in terms of the tasks/activities that they are responsible for) on a given project.
- Allows Project Managers to more effectively monitor and control these activities.

Therefore, a discipline in RUP is a collection of activities that are related to a major area of concern or field of study. Each activity is further decomposed into subactivities or one or many tasks. Tasks require an input artifact or artifacts for their successful execution, and these in turn produce or refine some form of output artifact(s). Note that these artifacts can include both document-based artifacts and executables. Each task has an associated role (or roles) responsible for performing that task. To provide additional support and guidance, each discipline in the base RUP offers a set of standard template artifacts related to that discipline. These artifacts, as well as the process, can be (and should be) customized/tailored for a given project or organization.

Discipline Workflow

RUP models the *when* as workflows, and each discipline in RUP has a workflow. Like other workflows, a discipline's workflow is a semi-ordered sequence of activities performed by specific roles to achieve a particular goal. This semi-ordered nature of discipline workflows emphasizes that they cannot present the nuances of scheduling "real work," because they cannot depict the optionality of activities or iterative nature of real projects. Yet, they still have value as a way for us to understand the process by breaking it into smaller areas of concerns.

Keep in mind that the RUP framework, which these workflows are part of, constitutes guidance on a rich set of software engineering principles. It is applicable to projects of different size and complexity, as well as to different development environments and domains. This means that no single project or organization will benefit from using all of RUP. Applying all of RUP will likely result in an inefficient project environment, where teams will struggle to keep focused on the important tasks and struggle to find the right set of information. Thus, as discussed earlier in this chapter, it is recommended that RUP be tailored to provide an appropriate and customized process for developing software. RUP tailoring and related tools are discussed in greater detail in Part IV of this book.

Unified Modelling Language

UML

Unified Modelling Language - **UML**

1. UML (Unified Modeling Language) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems.
2. UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997.
3. It was initially started to capture the behavior of complex software and non-software system and now it has become an OMG standard.

Unified Modelling Language - **UML**

OMG is continuously making efforts to create a truly industry standard.

- UML stands for **Unified Modeling Language**.
- UML is different from the other common programming languages such as C++, Java, COBOL, etc.
- UML is a pictorial language used to make software blueprints.
- UML can be described as a general purpose visual modeling language to visualize, specify, construct, and document software system.
- Although UML is generally used to model software systems, it is not limited within this boundary. It is also used to model non-software systems as well. For example, the process flow in a manufacturing unit, etc.

UML is not a programming language but tools can be used to generate code in various languages using UML diagrams. UML has a direct relation with object oriented analysis and design. After some standardization, UML has become an OMG standard.

Goals of UML

- ***A picture is worth a thousand words***, this idiom absolutely fits describing UML. Object-oriented concepts were introduced much earlier than UML. At that point of time, there were no standard methodologies to organize and consolidate the object-oriented development. It was then that UML came into picture.
- There are a number of goals for developing UML but the most important is to define some general purpose modeling language, which all modelers can use and it also needs to be made simple to understand and use.
- UML diagrams are not only made for developers but also for business users, common people, and anybody interested to understand the system. The system can be a software or non-software system. Thus it must be clear that UML is not a development method rather it accompanies with processes to make it a successful system.
- In conclusion, the goal of UML can be defined as a simple modeling mechanism to model all possible practical systems in today's complex environment.

A Conceptual Model of UML

To understand the conceptual model of UML, first we need to clarify what is a conceptual model? and why a conceptual model is required?

- A conceptual model can be defined as a model which is made of concepts and their relationships.
- A conceptual model is the first step before drawing a UML diagram. It helps to understand the entities in the real world and how they interact with each other.

As UML describes the real-time systems, it is very important to make a conceptual model and then proceed gradually. The conceptual model of UML can be mastered by learning the following three major elements –

- UML building blocks
- Rules to connect the building blocks
- Common mechanisms of UML

Object-Oriented Concepts

- UML can be described as the successor of object-oriented (OO) analysis and design.
- An object contains both data and methods that control the data. The data represents the state of the object. A class describes an object and they also form a hierarchy to model the real-world system. The hierarchy is represented as inheritance and the classes can also be associated in different ways as per the requirement.
- Objects are the real-world entities that exist around us and the basic concepts such as abstraction, encapsulation, inheritance, and polymorphism all can be represented using UML.
- UML is powerful enough to represent all the concepts that exist in object-oriented analysis and design. UML diagrams are representation of object-oriented concepts only. Thus, before learning UML, it becomes important to understand OO concept in detail.

Object-Oriented Concepts

Following are some fundamental concepts of the object-oriented world –

- **Objects** – Objects represent an entity and the basic building block.
- **Class** – Class is the blue print of an object.
- **Abstraction** – Abstraction represents the behavior of an real world entity.
- **Encapsulation** – Encapsulation is the mechanism of binding the data together and hiding them from the outside world.
- **Inheritance** – Inheritance is the mechanism of making new classes from existing ones.
- **Polymorphism** – It defines the mechanism to exists in different forms.

UML - **Building Blocks**

As UML describes the real-time systems, it is very important to make a conceptual model and then proceed gradually. The conceptual model of UML can be mastered by learning the following three major elements –

- UML building blocks
- Rules to connect the building blocks
- Common mechanisms of UML

This chapter describes all the UML building blocks. The building blocks of UML can be defined as –

- Things
- Relationships
- Diagrams

UML - **Things**

Things are the most important building blocks of UML.

There are 4 types of things:

- Structural
- Behavioral
- Grouping
- Annotational

UML - **Structural Things**

Structural things define the static part of the model.

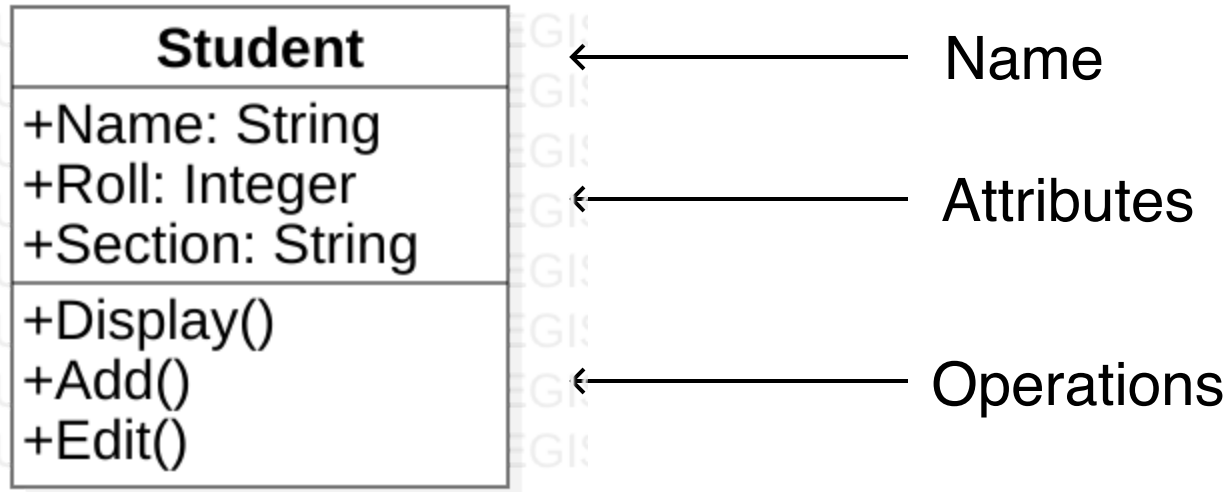
They represent the physical and conceptual elements.

Structural Things are:

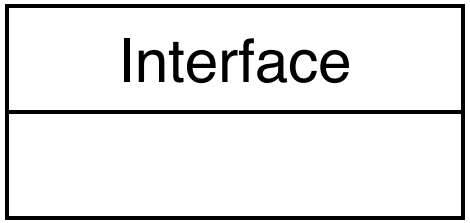
1. Class
2. Interface
3. Collaboration
4. Use-Case
5. Component
6. Node

Following are the brief descriptions of the structural things.

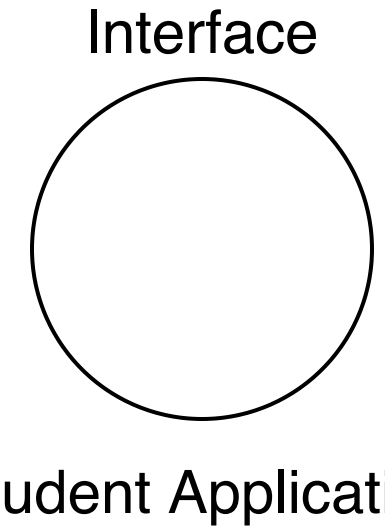
- **Class:** UML *Class* is represented by the following figure. The diagram is divided into 4 parts:
  - The top section is used to name the class.
  - The second section is used to show attributes of the class.
  - The third section is used to describe the operations of the class.
  - The final section is optional and used to show any additional components



- **Interface:** Interface defines a set of operations, which specify the responsibility of a class.

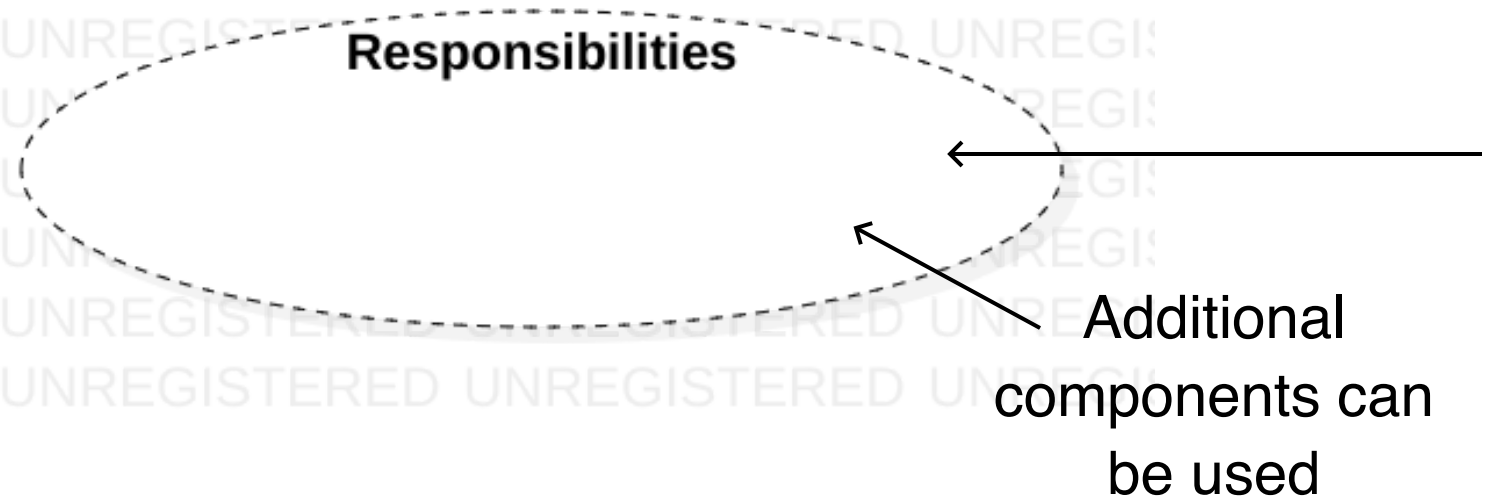


- **Interface Notation:** Interface is represented by a circle as shown in the following figure. It has a name which is generally written under the circle.



- Interfaces are used to describe the functionality without implementation.
- Interface is like a template where you define different functions, but not their implementation.

- **Collaboration:** Collaboration defines an interaction between elements.
- **Collaboration Notation:** Collaboration is represented by a dotted ellipse. It has a name written inside it.

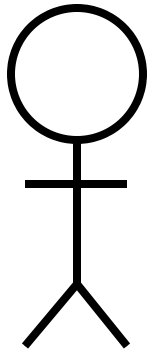


- Collaboration represents responsibilities. Generally, responsibilities are in a group.

- **Use-Case:** Use-Case represents a set of actions performed by a system for a specific goal.
- **Use-Case Notation:** Use-Case is represented by an ellipse with a name inside it.

- Use-Case is used to capture high level functions of a system.

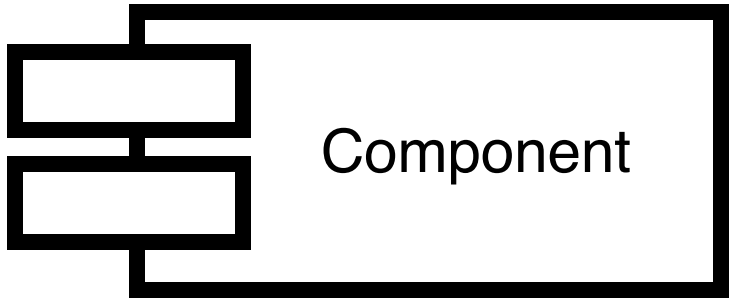
- **Actor Notation:** An actor is an internal or external entity who interacts with the system.



Administration

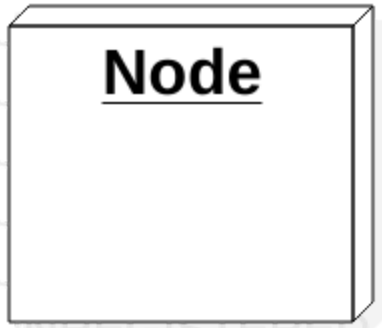
- Actors are used in Use-Case diagrams to describe the capabilities of various entities.

- **Component:** Component describes a physical part of the system.
- **Component Notation:** Components in UML are shown using the following figure. The name of the component is written inside it.



- Components can be used to represent any part of a system for which UML diagrams are made

- **Node:** A Node is a physical element that exists at runtime.
- **Node Notation:** Nodes in UML are represented using a cube. The name of the node is written inside the cube.



- Nodes are used to represent physical aspects of a system, like servers, switches, etc.

UML - **Behavioral Things**

A behavioral thing consists of the dynamic parts of UML models.

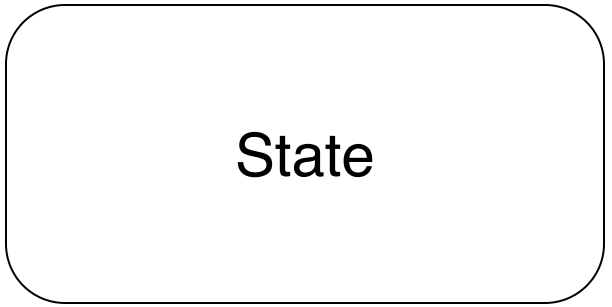
Listed below are all the behavioral things:

1. Interaction
2. State Machine

Following are the brief descriptions of the behavioral things.

- **Interaction:** Interaction is defined as a behavior that consists of a group of messages exchanged among elements to accomplish a specific task.

- **State Machine:** State machine is useful when the state of an object in its life cycle is important. It defines the sequence of states an object goes through in response to events. Events are external factors responsible for state change.

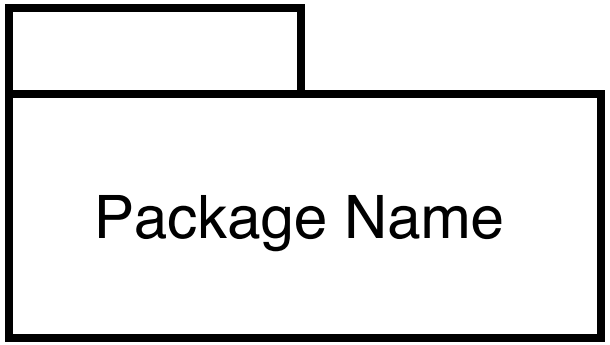


UML - **Grouping Things**

Grouping things can be defined as a mechanism to group elements of a UML model together.

There is only one grouping thing available:

- **Package:** Package is the only one grouping thing available for gathering structural and behavioral things.

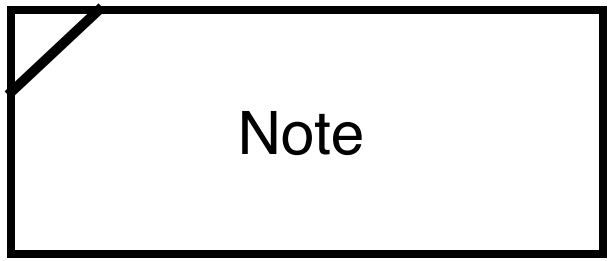


UML - **Annotational Things**

Annotational things can be defined as a mechanism to capture remarks, descriptions, and comments of UML model elements.

There is only one Annotational thing available:

- **Note:** It is the only one Annotational thing available. A note is used to render comments, constraints, etc. of an UML element.



UML - **Relationships**

Relationship is another most important building block of UML. It shows how the elements are associated with each other and this association describes the functionality of an application.

There are four kinds of relationships available.

1. Dependency
2. Association
3. Generalization
4. Realization

- **Dependency:** Dependency is a relationship between two things in which change in one element also affects the other.

Dependency

- **Association:** Association is basically a set of links that connects the elements of a UML model. It also describes how many objects are taking part in that relationship.

Association

- **Generalization:** Generalization can be defined as a relationship which connects a specialized element with a generalized element. It basically describes the inheritance relationship in the world of objects.

Generalization

- **Realization:** Realization can be defined as a relationship in which two elements are connected. One element describes some responsibility, which is not implemented and the other one implements them. This relationship exists in case of interfaces.

Realization

UML Diagrams

UML Diagrams

UML diagrams are the ultimate output of the entire discussion.

All the elements, relationships are used to make a complete UML diagram and the diagram represents a system.

The visual effect of the UML diagram is the most important part of the entire process. All the other elements are used to make it complete.

UML includes the following nine diagrams, the details of which are described in the subsequent chapters.

- Class diagram
- Object diagram
- Use case diagram
- Sequence diagram
- Collaboration diagram
- Activity diagram
- Statechart diagram
- Deployment diagram
- Component diagram

Class Diagrams

- Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.
- Class diagram describes the attributes and operations of a class and also the constraints imposed on the system. The class diagrams are widely used in the modeling of objectoriented systems because they are the only UML diagrams, which can be mapped directly with object-oriented languages.
- Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

Purpose of Class Diagrams

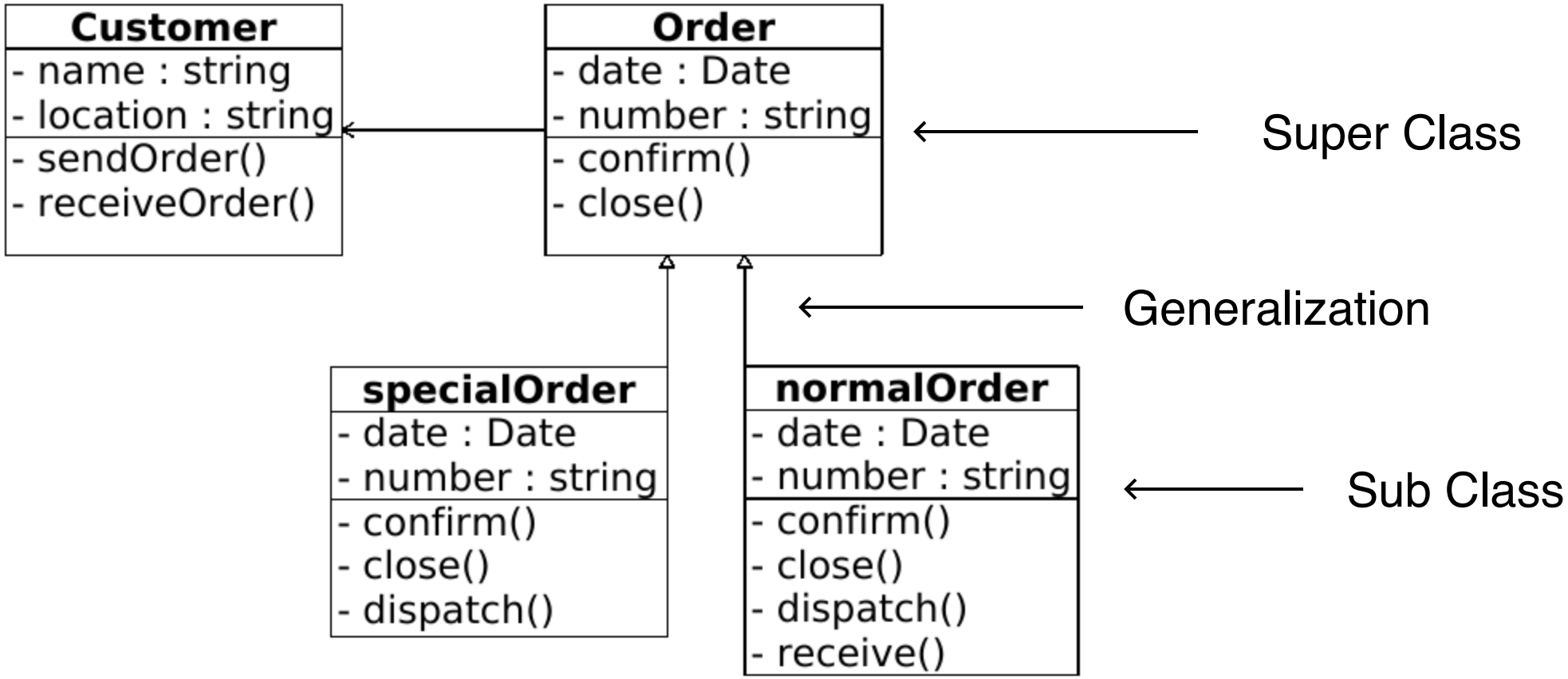
The purpose of class diagram is to model the static view of an application. Class diagrams are the only diagrams which can be directly mapped with object-oriented languages and thus widely used at the time of construction.

UML diagrams like activity diagram, sequence diagram can only give the sequence flow of the application, however class diagram is a bit different. It is the most popular UML diagram in the coder community.

The purpose of the class diagram can be summarized as –

- Analysis and design of the static view of an application.
- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering.

Example of Class Diagram



Object Diagrams

- Object diagrams are derived from class diagrams so object diagrams are dependent upon class diagrams.
- Object diagrams represent an instance of a class diagram. The basic concepts are similar for class diagrams and object diagrams. Object diagrams also represent the static view of a system but this static view is a snapshot of the system at a particular moment.
- Object diagrams are used to render a set of objects and their relationships as an instance.

Purpose of Object Diagrams

The purpose of a diagram should be understood clearly to implement it practically. The purposes of object diagrams are similar to class diagrams.

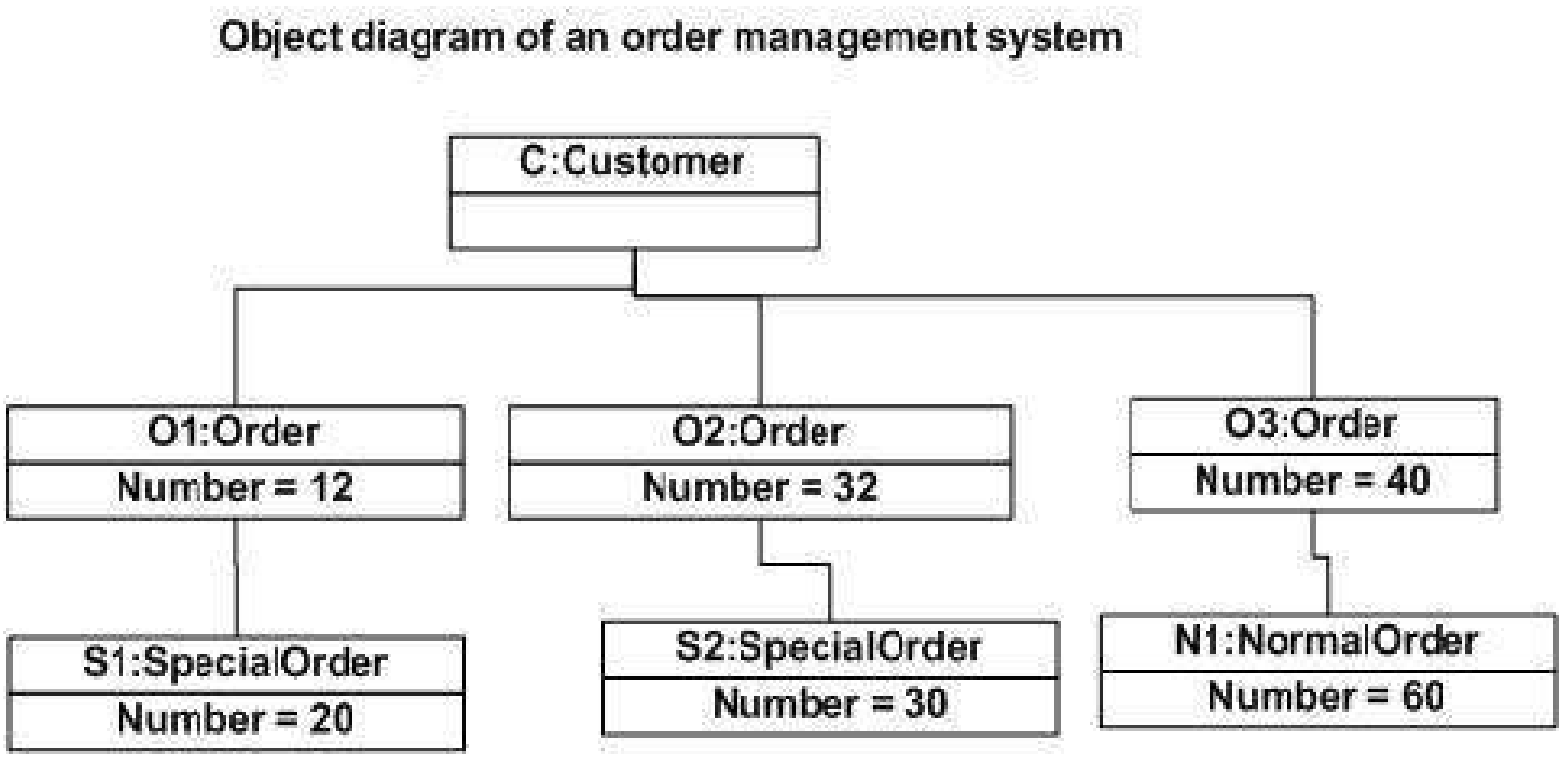
The difference is that a class diagram represents an abstract model consisting of classes and their relationships. However, an object diagram represents an instance at a particular moment, which is concrete in nature.

It means the object diagram is closer to the actual system behavior. The purpose is to capture the static view of a system at a particular moment.

The purpose of the object diagram can be summarized as –

- Forward and reverse engineering.
- Object relationships of a system
- Static view of an interaction.
- Understand object behaviour and their relationship from practical perspective

Example of Object Diagram



Component Diagrams

- Component diagrams are different in terms of nature and behavior. Component diagrams are used to model the **physical aspects** of a system.
- **Physical aspects** are the elements such as executables, libraries, files, documents, etc. which reside in a node.
- Component diagrams are used to visualize the organization and relationships among components in a system. These diagrams are also used to make executable systems.

Purpose of Component Diagrams

Component diagram is a special kind of diagram in UML. The purpose is also different from all other diagrams discussed so far. It does not describe the functionality of the system but it describes the components used to make those functionalities.

Thus from that point of view, component diagrams are used to visualize the physical components in a system. These components are libraries, packages, files, etc.

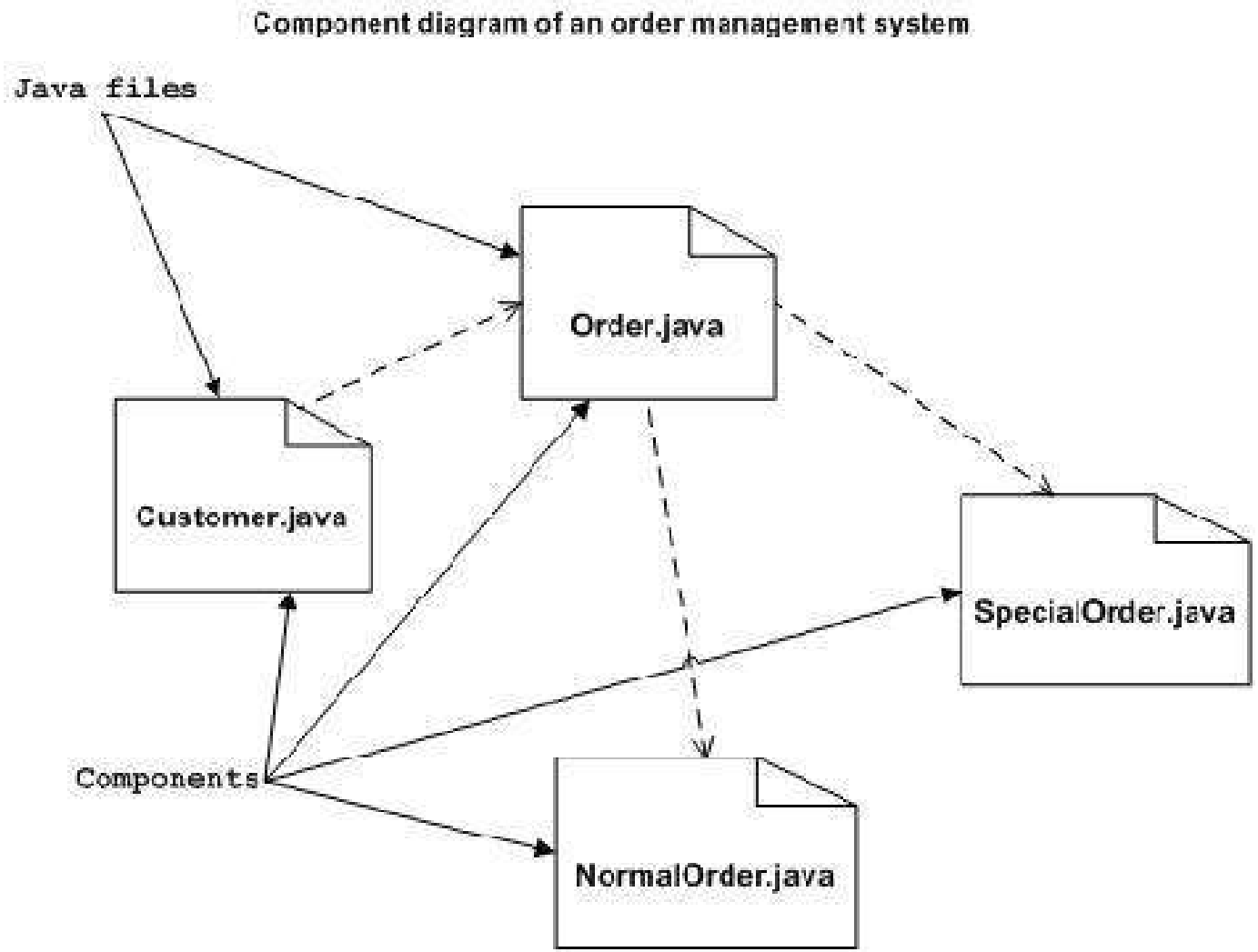
Component diagrams can also be described as a static implementation view of a system. Static implementation represents the organization of the components at a particular moment.

A single component diagram cannot represent the entire system but a collection of diagrams is used to represent the whole.

The purpose of the component diagram can be summarized as –

- Visualize the components of a system.
- Construct executables by using forward and reverse engineering.
- Describe the organization and relationships of the components.

Example of Component Diagram



Deployment Diagrams

- Deployment diagrams are used to visualize the topology of the physical components of a system, where the software components are deployed.
- Deployment diagrams are used to describe the static deployment view of a system. Deployment diagrams consist of nodes and their relationships.

Purpose of Deployment Diagrams

The term Deployment itself describes the purpose of the diagram. Deployment diagrams are used for describing the hardware components, where software components are deployed. Component diagrams and deployment diagrams are closely related.

Component diagrams are used to describe the components and deployment diagrams shows how they are deployed in hardware.

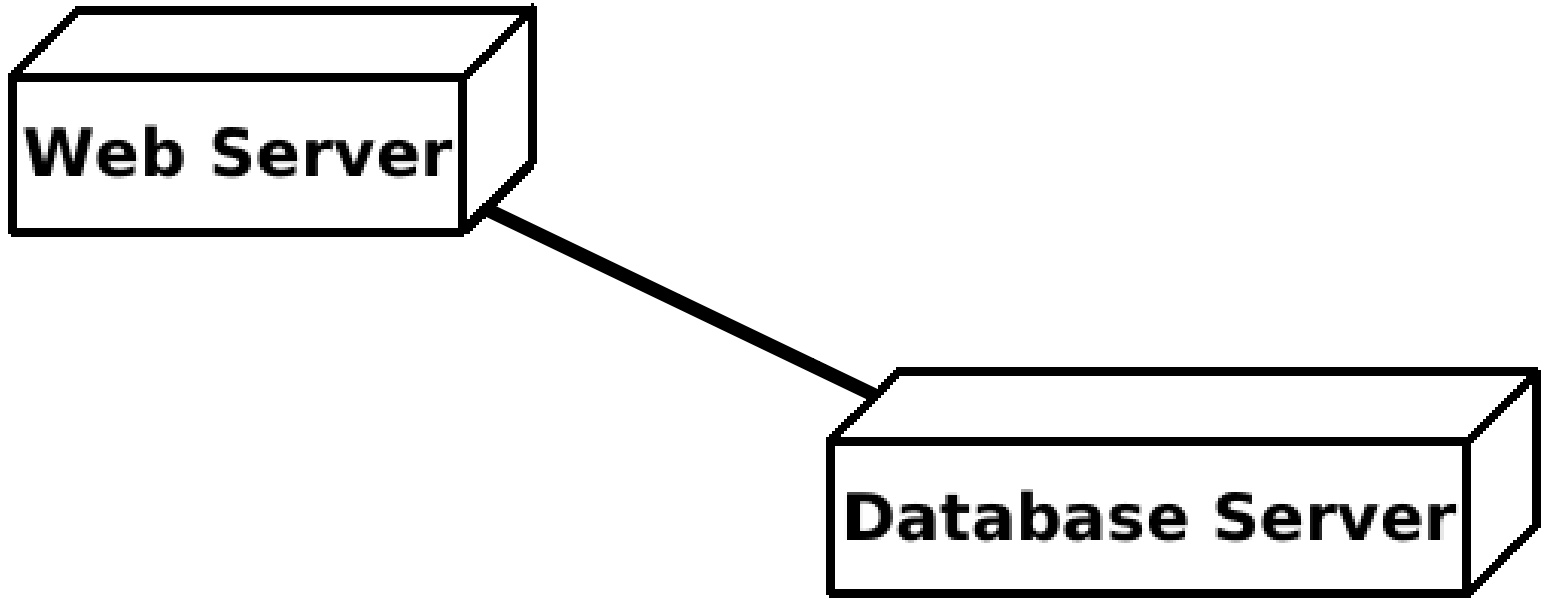
UML is mainly designed to focus on the software artifacts of a system. However, these two diagrams are special diagrams used to focus on software and hardware components.

Most of the UML diagrams are used to handle logical components but deployment diagrams are made to focus on the hardware topology of a system. Deployment diagrams are used by the system engineers.

The purpose of deployment diagrams can be described as –

- Visualize the hardware topology of a system.
- Describe the hardware components used to deploy software components.
- Describe the runtime processing nodes.

Example of Deployment Diagram



Use-Case Diagrams

- To model a system, the most important aspect is to capture the dynamic behavior. Dynamic behavior means the behavior of the system when it is running/operating.
- Only static behavior is not sufficient to model a system rather dynamic behavior is more important than static behavior. In UML, there are five diagrams available to model the dynamic nature and use case diagram is one of them. Now as we have to discuss that the use case diagram is dynamic in nature, there should be some internal or external factors for making the interaction.
- These internal and external agents are known as actors. Use case diagrams consists of actors, use cases and their relationships. The diagram is used to model the system/subsystem of an application. A single use case diagram captures a particular functionality of a system.
- Hence to model the entire system, a number of use case diagrams are used.

Purpose of Use-Case Diagrams

The purpose of use case diagram is to capture the dynamic aspect of a system. However, this definition is too generic to describe the purpose, as other four diagrams (activity, sequence, collaboration, and Statechart) also have the same purpose. We will look into some specific purpose, which will distinguish it from other four diagrams.

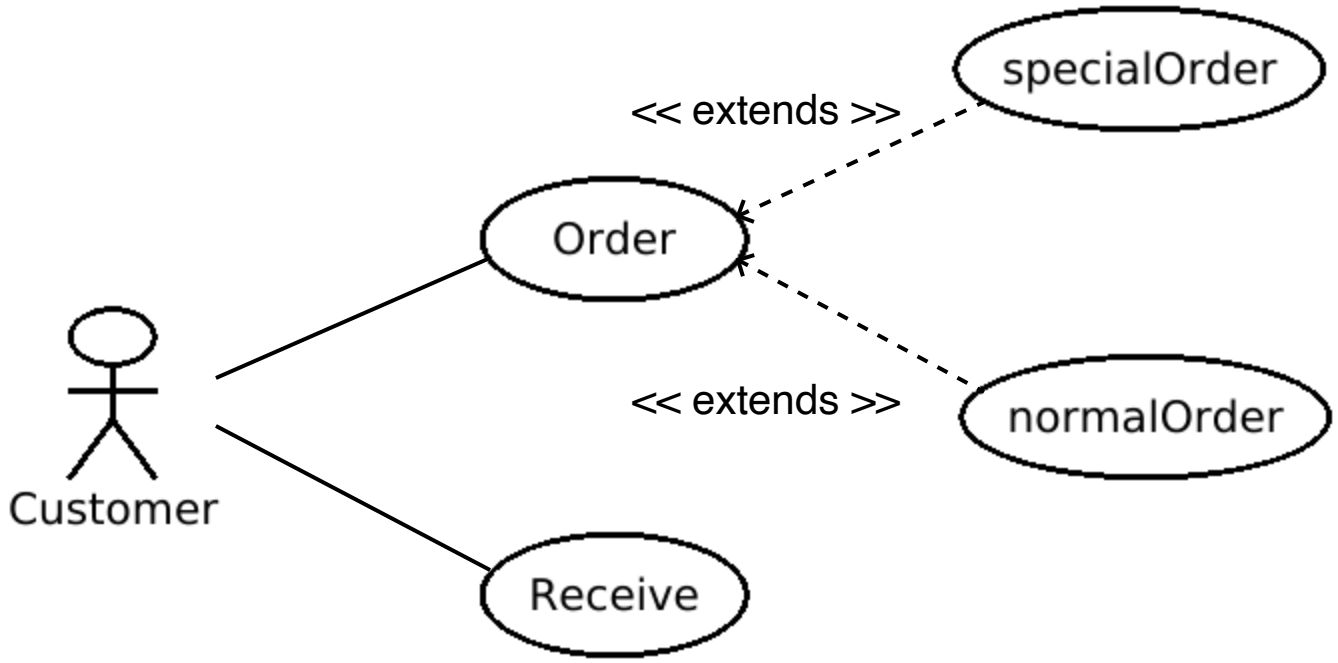
Use case diagrams are used to gather the requirements of a system including internal and external influences. These requirements are mostly design requirements. Hence, when a system is analyzed to gather its functionalities, use cases are prepared and actors are identified.

When the initial task is complete, use case diagrams are modelled to present the outside view.

In brief, the purposes of use case diagrams can be said to be as follows –

- Used to gather the requirements of a system.
- Used to get an outside view of a system.
- Identify the external and internal factors influencing the system.
- Show the interaction among the requirements are actors.

Example of Use-Case Diagram



Interaction Diagrams

- From the term Interaction, it is clear that the diagram is used to describe some type of interactions among the different elements in the model. This interaction is a part of dynamic behavior of the system.
- This interactive behavior is represented in UML by two diagrams known as **Sequence diagram** and **Collaboration diagram**. The basic purpose of both the diagrams are similar.
- Sequence diagram emphasizes on time sequence of messages and collaboration diagram emphasizes on the structural organization of the objects that send and receive messages.

Purpose of Interaction Diagrams

The purpose of interaction diagrams is to visualize the interactive behavior of the system. Visualizing the interaction is a difficult task. Hence, the solution is to use different types of models to capture the different aspects of the interaction.

Sequence and collaboration diagrams are used to capture the dynamic nature but from a different angle.

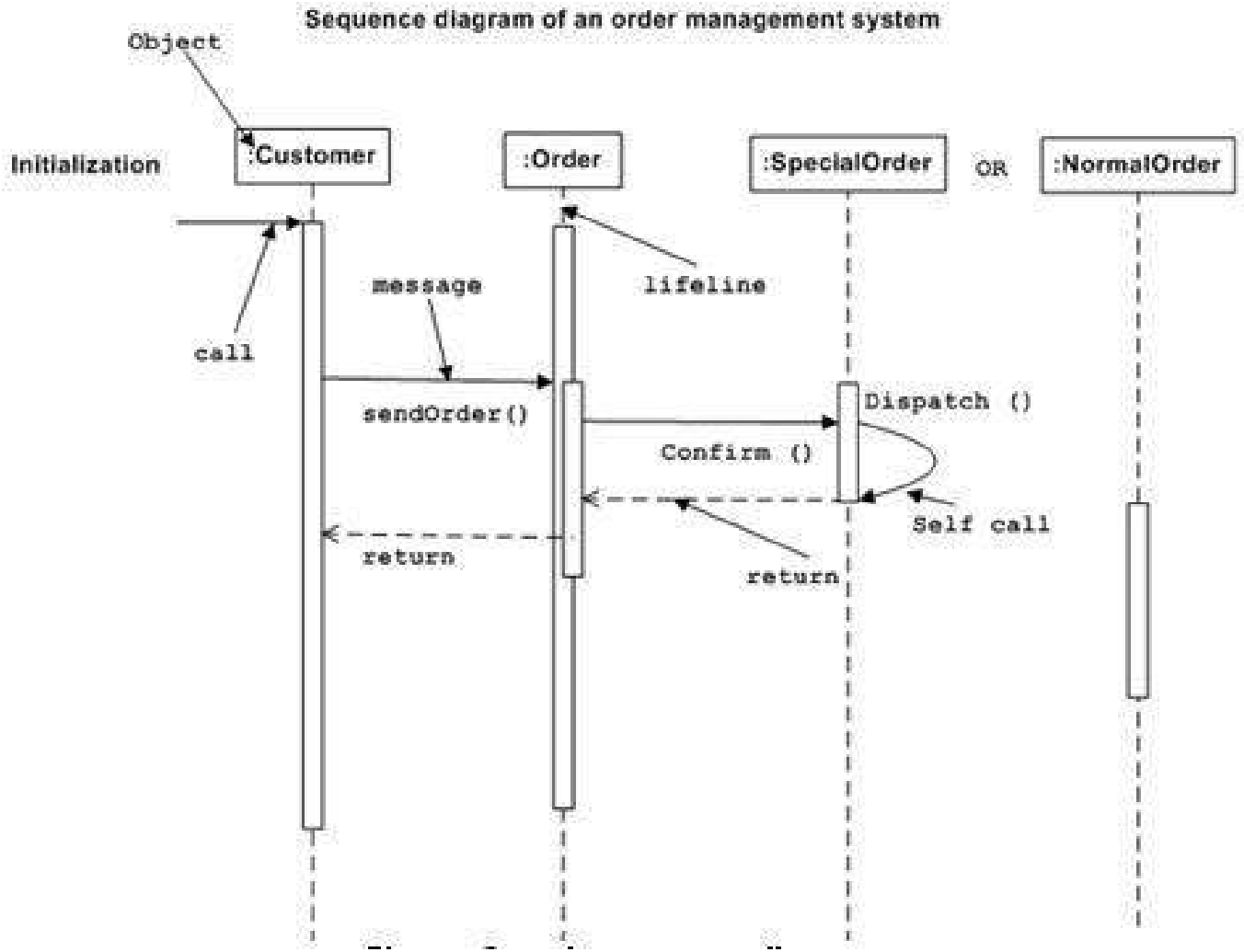
The purpose of interaction diagram is –

- To capture the dynamic behaviour of a system.
- To describe the message flow in the system.
- To describe the structural organization of the objects.
- To describe the interaction among objects.

Sequence Diagram (Example)

- The sequence diagram has four objects (Customer, Order, SpecialOrder and NormalOrder).
- The following diagram shows the message sequence for *SpecialOrder* object and the same can be used in case of *NormalOrder* object. It is important to understand the time sequence of message flows. The message flow is nothing but a method call of an object.
- The first call is *sendOrder ()* which is a method of *Order object*. The next call is *confirm ()* which is a method of *SpecialOrder* object and the last call is *Dispatch ()* which is a method of *SpecialOrder* object. The following diagram mainly describes the method calls from one object to another, and this is also the actual scenario when the system is running.

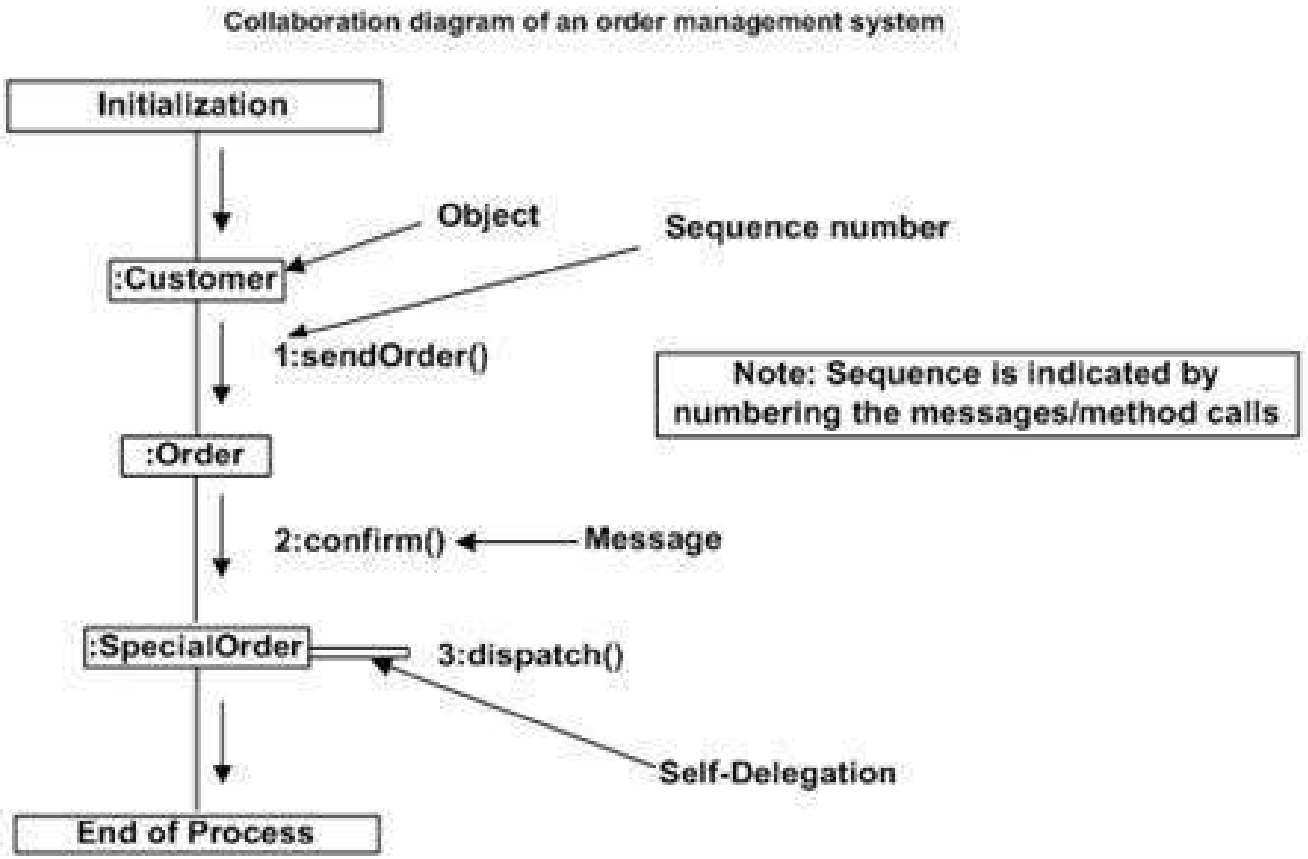
Sequence Diagram (Example)



Collaboration Diagram (Example)

- The second interaction diagram is the collaboration diagram. It shows the object organization as seen in the following diagram. In the collaboration diagram, the method call sequence is indicated by some numbering technique. The number indicates how the methods are called one after another. We have taken the same order management system to describe the collaboration diagram.
- Method calls are similar to that of a sequence diagram. However, difference being the sequence diagram does not describe the object organization, whereas the collaboration diagram shows the object organization.
- To choose between these two diagrams, emphasis is placed on the type of requirement. If the time sequence is important, then the sequence diagram is used. If organization is required, then collaboration diagram is used.

Collaboration Diagram (Example)



Statechart Diagrams

- The name of the diagram itself clarifies the purpose of the diagram and other details. It describes different states of a component in a system. The states are specific to a component/object of a system.
- A Statechart diagram describes a state machine. State machine can be defined as a machine which defines different states of an object and these states are controlled by external or internal events.
- Activity diagram explained in the next chapter, is a special kind of a Statechart diagram. As Statechart diagram defines the states, it is used to model the lifetime of an object.

Purpose of Statechart Diagrams

Statechart diagram is one of the five UML diagrams used to model the dynamic nature of a system. They define different states of an object during its lifetime and these states are changed by events. Statechart diagrams are useful to model the reactive systems. Reactive systems can be defined as a system that responds to external or internal events.

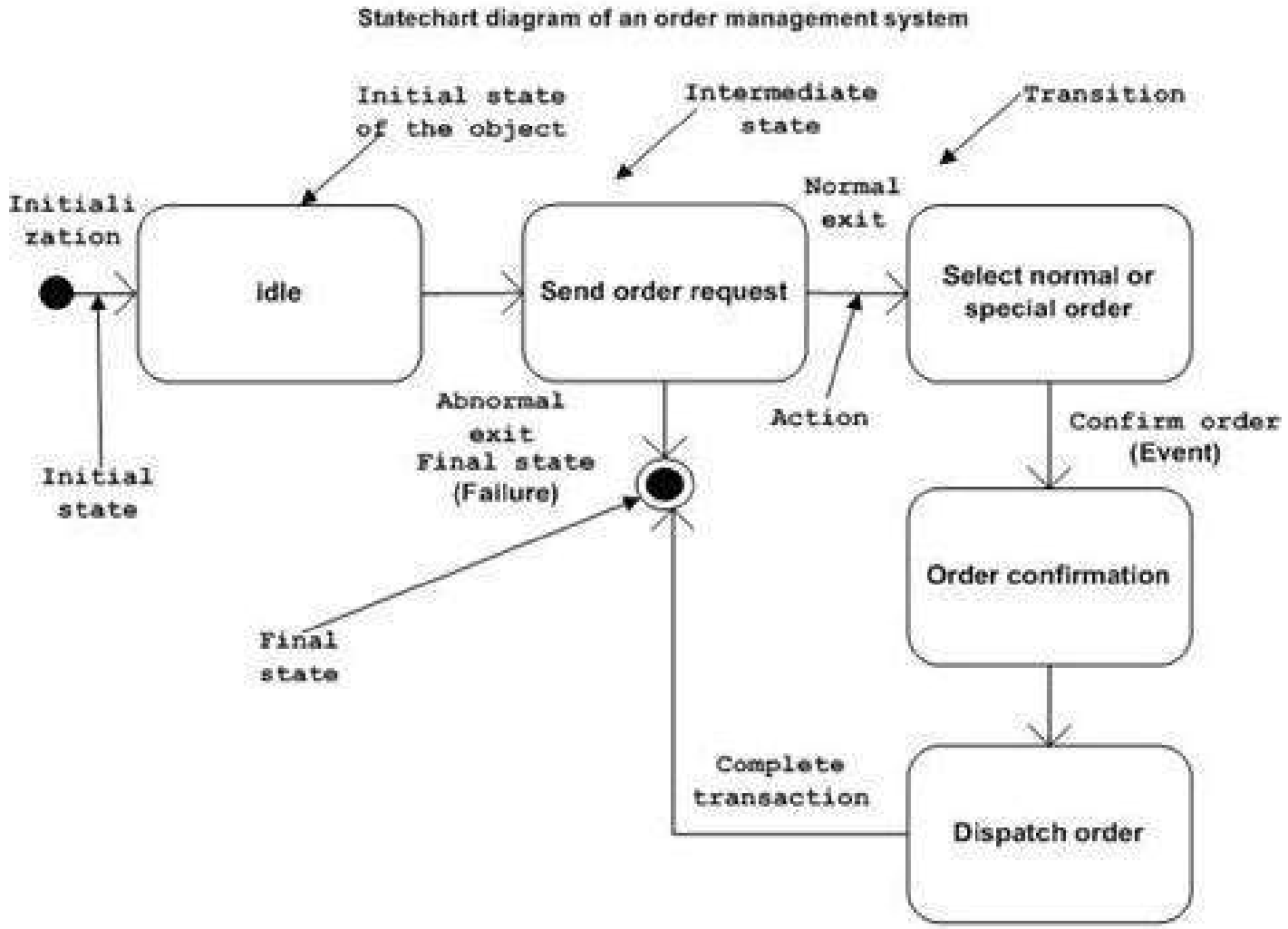
Statechart diagram describes the flow of control from one state to another state. States are defined as a condition in which an object exists and it changes when some event is triggered. The most important purpose of Statechart diagram is to model lifetime of an object from creation to termination.

Statechart diagrams are also used for forward and reverse engineering of a system. However, the main purpose is to model the reactive system.

Following are the main purposes of using Statechart diagrams –

- To model the dynamic aspect of a system.
- To model the life time of a reactive system.
- To describe different states of an object during its life time.
- Define a state machine to model the states of an object.

Example of Statechart Diagram



Activity Diagrams

- Activity diagram is another important diagram in UML to describe the dynamic aspects of the system.
- Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.
- The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc

Purpose of Activity Diagrams

The basic purposes of activity diagrams is similar to other four diagrams. It captures the dynamic behavior of the system. Other four diagrams are used to show the message flow from one object to another but activity diagram is used to show message flow from one activity to another.

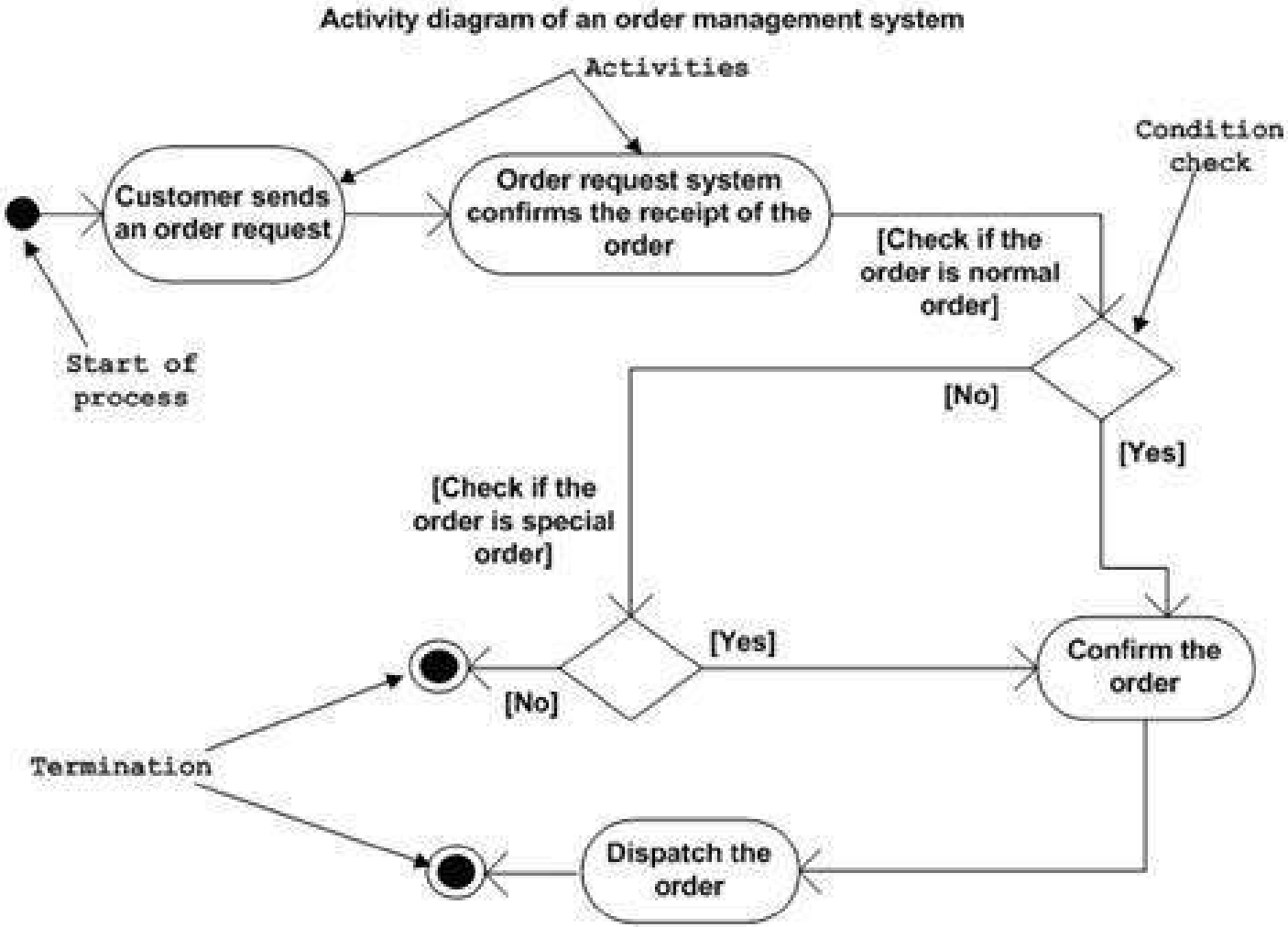
Activity is a particular operation of the system. Activity diagrams are not only used for visualizing the dynamic nature of a system, but they are also used to construct the executable system by using forward and reverse engineering techniques. The only missing thing in the activity diagram is the message part.

It does not show any message flow from one activity to another. Activity diagram is sometimes considered as the flowchart. Although the diagrams look like a flowchart, they are not. It shows different flows such as parallel, branched, concurrent, and single.

The purpose of an activity diagram can be described as –

- Draw the activity flow of a system.
- Describe the sequence from one activity to another.
- Describe the parallel, branched and concurrent flow of the system.

Example of Activity Diagram



Quiz

Which of the following are behavior diagrams?

1. Use-Case Diagram
2. Class Diagram
3. Statechart Diagram
4. Sequence Diagram

A car has a Starter, some Lights, an AirConditioning System, some Wheels. The driver operates first the Starter, then the Lights, then the AirConditioning System, and finally the Wheels. Which pair of diagrams is most useful to model this situation?

1. Use-Case and Class Diagram
2. Class and Sequence Diagram
3. Statechart and Class Diagram
4. Sequence and Object Diagram

Activity - 1

Propose a use case diagram for an ATM machine for withdrawing cash.

Make the use case simple yet informative; only include the major features.

Activity - 2

Propose a sequence diagram for an ATM machine for withdrawing cash.

Activity - 3

Propose a Use-Case diagram for a vending machine

for buying snacks and drinks.

Keep in mind that vending machines will require maintenance from time to time.

Activity - 4

Propose a sequence diagram for a vending machine for buying snacks and drinks.

Activity - 5

Propose a class diagram for a singly linked list